

pycache

```
cart.py  x  main.py  product.py
cart.py > ...
1  from product import Product
2
3
4  class ShoppingCart:
5      ... def __init__(self):
6      ...     self.__products = []
7      ...     self.__cart = []
8
9      ... def add_product(self, pid, name, price):
10     ...     self.__products.append(Product(pid, name, price))
11     ...     print("Product added successfully!")
12
13     def view_products(self):
14         if not self.__products:
15             print("No products available.")
16             return
17
18         print("\n===== PRODUCTS =====")
19
20         for product in self.__products:
21             product.display()
22
23     def add_to_cart(self, pid, qty):
24         for product in self.__products:
25             if product.get_id() == pid:
26                 self.__cart.append((product, int(qty)))
27                 print("Added to cart!")
28                 return
29
30         print("Product not found!")
31
32     def view_cart(self):
33         if not self.__cart:
34             print("Cart is empty.")
35             return
36
37         subtotal = 0
38
39         print("\n===== SHOPPING CART =====")
40
41         for product, qty in self.__cart:
42             total = product.get_price() * qty
43             subtotal += total
44             print(f"{product.get_name()} x{qty} = Rs. {total}")
45
46         discount = subtotal * 0.10 if subtotal >= 5000 else 0
47         vat = (subtotal - discount) * 0.13
48         grand_total = subtotal - discount + vat
49
50         print("-----")
51         print(f"Subtotal : Rs. {subtotal}")
52         print(f"Discount : Rs. {discount}")
53         print(f"VAT      : Rs. {vat}")
54         print(f"Total    : Rs. {grand_total}")
55
56     def checkout(self):
57         if not self.__cart:
58             print("Cart is empty.")
59             return
60
61         self.view_cart()
62         print("\nCheckout Successful!")
63         self.__cart.clear()
```

[main.py](#)

```
main.py > ...
1  from cart import ShoppingCart
2
3
4  def menu():
5      cart = ShoppingCart()
6
7      while True:
8          print("\n" + "=" * 40)
9          print("      E-COMMERCE CART")
10         print("=" * 40)
11         print("1. Add Product")
12         print("2. View Products")
13         print("3. Add To Cart")
14         print("4. View Cart")
15         print("5. Checkout")
16         print("6. Exit")
17
18         choice = input("Enter your choice: ")
19
20         if choice == "1":
21             pid = input("Product ID: ")
22             name = input("Product Name: ")
23             price = input("Price: ")
24
25             cart.add_product(pid, name, price)
26
27         elif choice == "2":
28             cart.view_products()
29
30         elif choice == "3":
31             pid = input("Product ID: ")
32             qty = input("Quantity: ")
33
34             cart.add_to_cart(pid, qty)
35
36         elif choice == "4":
37             cart.view_cart()
38
39         elif choice == "5":
40             cart.checkout()
41
42         elif choice == "6":
43             print("Thank you for shopping!")
44             break
45
46         else:
47             print("Invalid choice!")
48
49
50 if __name__ == "__main__":
51     menu()
```

product.py

```
product.py > Product
1 class Product:
2     def __init__(self, product_id, name, price):
3         self.__product_id = product_id
4         self.__name = name
5         self.__price = float(price)
6
7     def get_id(self):
8         return self.__product_id
9
10    def get_name(self):
11        return self.__name
12
13    def get_price(self):
14        return self.__price
15
16    def display(self):
17        print(f"{self.__product_id} | {self.__name} | Rs. {self.__price}")
```

File 1: product.py

Purpose

The `product.py` file defines the **Product** class, which stores information about each product, including its ID, name, and price.

Functions

- `__init__()` – Initializes the product ID, name, and price.
- `get_id()` – Returns the product ID.
- `get_name()` – Returns the product name.
- `get_price()` – Returns the product price.
- `display()` – Displays the product details in a readable format.

Summary

This file acts as the product model and provides the necessary methods to access product information safely.

File 2: cart.py

Purpose

The `cart.py` file contains the **ShoppingCart** class, which manages products, shopping cart operations, billing, and checkout.

Functions

- `add_product()` – Adds a new product to the product list.
- `view_products()` – Displays all available products.
- `add_to_cart()` – Adds a selected product and quantity to the cart.
- `view_cart()` – Calculates the subtotal, discount, VAT, and total bill.
- `checkout()` – Displays the final bill and clears the cart after purchase.

Summary

This file performs the main operations of the shopping cart system, including managing products and calculating the final payment.

File 3: `main.py`

Purpose

The `main.py` file is the main program that provides a menu-driven interface for users to interact with the shopping cart system.

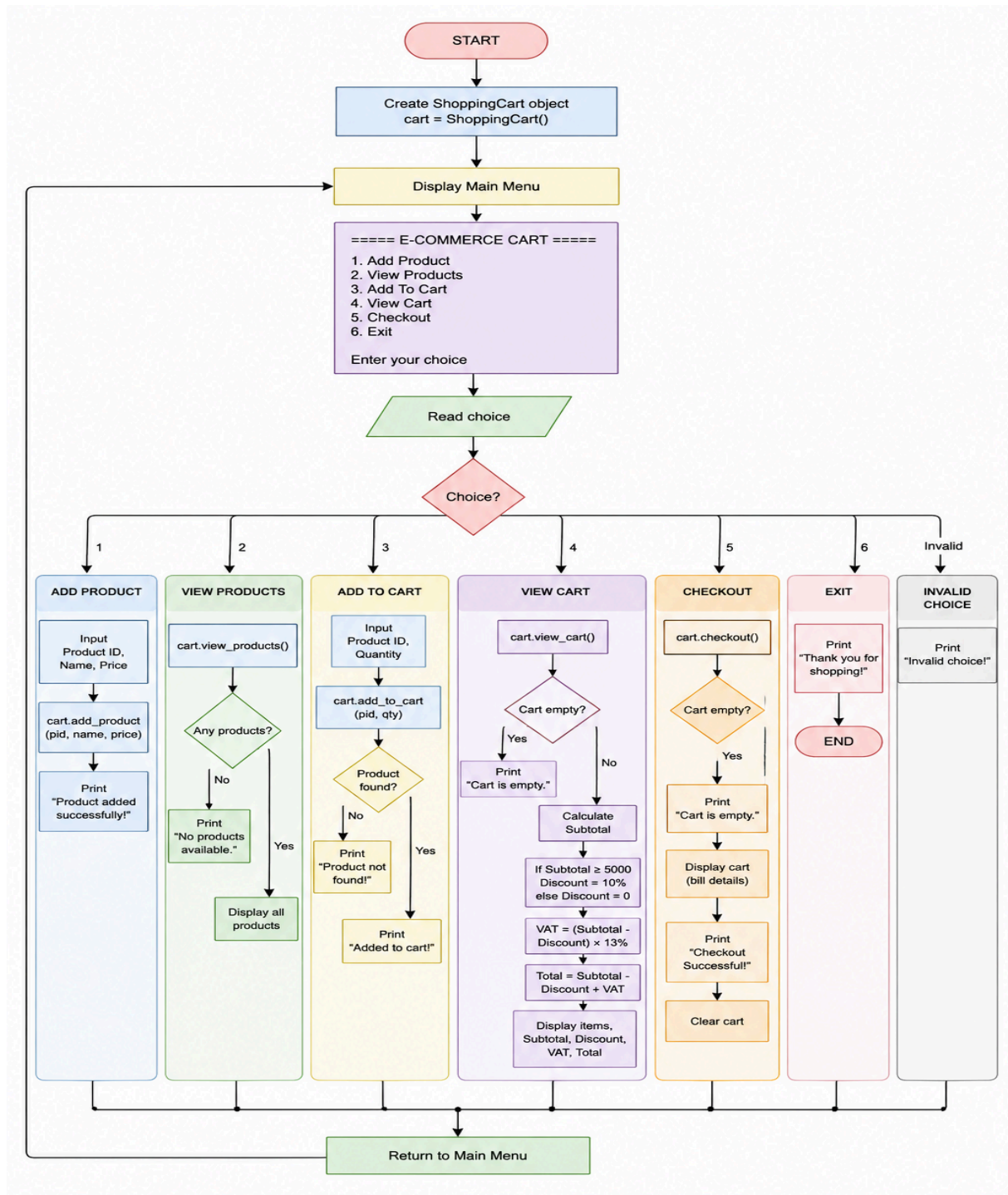
Working

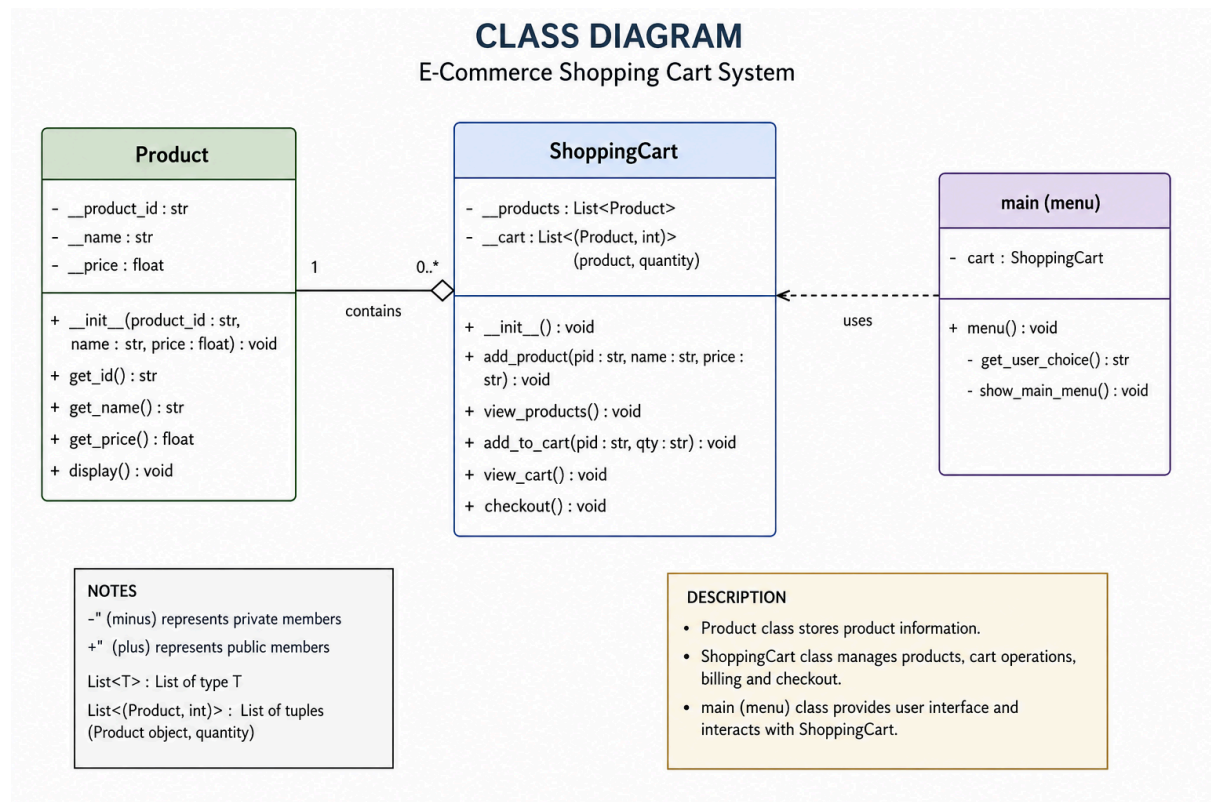
- Creates a **ShoppingCart** object.
- Displays menu options to the user.
- Accepts user input and performs actions such as adding products, viewing products, adding items to the cart, viewing the cart, checking out, or exiting the program.
- Calls the appropriate methods from the **ShoppingCart** class based on the user's choice.

Summary

This file controls the execution of the program and connects the user with the shopping cart functionalities through a simple menu interface.

FLOWCHART:





File Structure

E-Commerce Shopping Cart System

```
|
|—— main.py    → Main program (Menu)
|—— cart.py    → Shopping cart operations
|—— product.py → Product class
|—— __pycache__ → Compiled Python files
```

Conclusion

The E-Commerce Shopping Cart System is a simple Python application developed using object-oriented programming principles. It is divided into three main files: **product.py** stores product information, **cart.py** manages shopping cart operations and billing, and **main.py** provides a menu-driven interface for user interaction. The program allows users to add products, view available products, add items to the cart, calculate discounts and VAT, and complete the checkout process. This modular structure makes the system easy to understand, maintain, and extend with additional features in the future.